



***SecGraph: Enterprise Identity and Access Risk
Analyzer***

by

MEERA V NAIR

(2539746)

Under the guidance of

Dr. Priya Stella Mary I

Assistant Professor

Department of Computer Science

CHRIST (Deemed to be University)

Bangalore Yeshwanthpur Campus



A Project report submitted in partial fulfilment of the requirements for the award of
degree of Master of Artificial Intelligence & Cybersecurity of CHRIST (Deemed to be
University)

SEPTEMBER – 2026

CERTIFICATE

*It is certified that this project report entitled "**SecGraph: Enterprise Identity and Access Risk Analyzer**" has been submitted to CHRIST (Deemed to be University), Bangalore, as project work, as partial fulfilment of the requirement for the award of the degree of **MASTER OF SCIENCE** in Artificial Intelligence and Cyber Security. This work has been completed by MEERA V NAIR (Reg. No. 2539746) under my supervision and she has put in the required attendance.*

Head of the Department

Project Guide

Valued-by:

1. _____

Name: Meera V Nair

Register Number: 2539746

Examination Centre: CHRIST (Deemed to

be University)

2. _____

Date of Exam:



DECLARATION

*I hereby certify that the work presented in this project report is my own genuine work and has been carried out by me under the supervision of **Dr. Priya Stella Mary I, Assistant Professor, Department of Computer Science, CHRIST (Deemed to be University), Bangalore Yeshwanthpur Campus.** The work embodied in this project report has not been submitted for a degree or diploma in any other University.*

Date:

Place: Bangalore

MEERA V NAIR

(Reg. No. 2539746)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who have contributed to the successful completion of my Specialization Project. Their support and guidance have been invaluable, and I am deeply thankful for their assistance throughout this journey. First and foremost, I would like to extend my heartfelt thanks to Dr. Fr Joseph C C, Vice Chancellor, Dr. Fr. Benny Thomas, Campus Director, Associate Director, Dr. Fr. Biju K Chacko, Dr. Joby Thomas, Dean, CHRIST (Deemed to be University) Yeshwanthpur Campus for letting us carry our specialization project of M.Sc Artificial Intelligence and Cyber Security in our Campus.

I would like to extend my heartfelt thanks to the HOD Dr. Vinay M and Associate HOD Dr .C.Balakrishnan of the Computer Science department of CHRIST (Deemed to be University), Bangalore Yeshwanthpur Campus, for fostering a conducive learning environment that has been instrumental in shaping my academic experience.

I am also immensely grateful to my project guide, ***Dr. Priya Stella Mary I*** for her expertise mentorship, and unwavering support throughout this project. Your patience and dedication to my growth as a data scientist have been instrumental in the successful completion of this project.

I would like to thank all the faculty members of the Department of Computer Science for their help to complete the project work successfully. Also, I would like to thank my fellow students and colleagues for their camaraderie, which made this academic journey both enjoyable and enriching. Last but not least, I want to express my gratitude to my family and friends for their unwavering support, understanding, and encouragement throughout this challenging but rewarding journey. Your belief in me has been a constant source of motivation.

In conclusion, this project would not have been possible without the guidance and support of the individuals mentioned above and the collective effort of everyone who played a part in my academic pursuit. Thank you all for your contributions, and I am deeply appreciative of your help.

MEERA V NAIR
(2539746)

ABSTRACT

Enterprise environments generate large volumes of identity, authentication, access, and resource-usage data. Conventional security-monitoring approaches often analyze these events independently or rely primarily on predefined rules, making it difficult to identify complex relationships and unusual behavioural patterns associated with potentially risky identities. This project proposes SecGraph, an explainable hybrid analytics system for enterprise identity risk assessment. The system represents relationships among users, roles, permissions, and resources as a graph and applies graph analytics to identify structurally significant identities and access relationships. Behavioural features extracted from activity logs are analyzed using an unsupervised anomaly-detection approach based on Isolation Forest, and rule-based indicators identify conditions such as excessive permissions and sensitive-resource access. The outputs of graph analysis, behavioural analysis, and rule-based analysis are combined into a configurable hybrid risk score that categorizes identities into low-, medium-, and high-risk levels, with explainable reasons, a visual dashboard, alerts, and generated reports. The system is designed as a research prototype to support authorized security analysts in prioritizing identity-risk investigations rather than automatically determining malicious intent.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION	1
1.1 Problem Description	1
1.2 Background	1
1.3 Motivation	2
1.4 Problem Statement	2
1.5 Aim	3
1.6 Objectives	3
1.7 Research Questions	4
1.8 Scope of the Project	4
1.9 Significance of the Project	5
1.10 Project Methodology	5
CHAPTER 2: SYSTEM ANALYSIS	7
2.1 Functional Specifications	7
2.2 Non-Functional Requirements	8
2.3 Existing System	9
2.4 Limitations of Existing System	9
2.5 Proposed System	9
2.6 Advantages of Proposed System	10
2.7 Feasibility Study	10
2.8 Block Diagram	11
2.9 System Requirements	12
CHAPTER 3: SYSTEM DESIGN	13
3.1 System Architecture	13

3.2 Module Design	14
3.3 Database Design	15
3.4 System Configuration	24
3.5 Interface Design	24
3.6 Risk Scoring Design	25
3.7 Graph Model Design	26
3.8 Machine Learning Design	27
3.9 Reports Design	27
CHAPTER 4: IMPLEMENTATION	28
4.1 Coding Standard	28
4.2 Development Environment	28
4.3 Dataset Preparation	28
4.4 Data Processing	29
4.5 Feature Engineering	29
4.6 Graph Construction	30
4.7 Graph Analytics	30
4.8 Behavioural Analytics	31
4.9 Anomaly Detection	31
4.10 Rule Engine	31
4.11 Hybrid Risk Engine	32
4.12 Explainability Module	32
4.13 Alert Module	32
4.14 Dashboard Implementation	32
4.15 Report Generation	34

4.16 Security Implementation	35
CHAPTER 5: TESTING AND RESULTS	36
5.1 Testing Strategy	36
5.2 Unit Testing	36
5.3 Integration Testing	36
5.4 System Testing	36
5.5 Test Cases	37
5.6 Test Reports	38
CHAPTER 6: CONCLUSION	39
6.1 Summary	39
6.2 Design and Implementation Issues	39
6.3 Advantages	40
6.4 Limitations	40
6.5 Future Enhancements	40
6.6 Final Conclusion	41
APPENDIX A — USER MANUAL	42
APPENDIX B — DATASET INFORMATION	43
APPENDIX C — SAMPLE OUTPUTS	44
REFERENCES	54

LIST OF TABLES

Table 2.1 — Functional Requirements	7
Table 2.2 — Non-Functional Requirements	8
Table 2.3 — Hardware Requirements	12
Table 2.4 — Software Requirements	12
Table 3.1 — Admin: Table Structure	15
Table 3.2 — Users: Table Structure	15
Table 3.3 — Login Events: Table Structure	16
Table 3.4 — Resource Access: Table Structure	16
Table 3.5 — Behaviour Features: Table Structure	17
Table 3.6 — Graph Metrics: Table Structure	18
Table 3.7 — Risk Results: Table Structure	19
Table 3.8 — Alert: Table Structure	20
Table 3.9 — Analysis History: Table Structure	20
Table 3.10 — System Settings: Table Structure	22
Table 3.11 — Risk Classification Bands	26
Table 4.1 — CERT r4.2 Dataset Inventory & Attributes	28
Table 4.2 — Summary of Engineered Behavioral Features	29
Table 4.3 — Representative Graph Metrics Output	31
Table 5.1 — Comprehensive Unit and System Test Cases	37
Table 5.2 — Test Execution Summary	38

LIST OF FIGURES

Fig. 1.1 — SecGraph Project Methodology	6
Fig. 2.1 — SecGraph System Block Diagram	11
Fig. 3.1 — SecGraph 4-Layer System Architecture	13
Fig. 3.2 — Data Flow Diagram (DFD) Level 0 (Context)	22
Fig. 3.3 — Data Flow Diagram (DFD) Level 1 (Analytical Pipeline)	23
Fig. 3.4 — Entity Relationship (ER) Diagram	23
Fig. 3.5 — Application Execution Flow Chart	25
Fig. 4.1 — SecGraph Split-Screen Cyber Login Screen	33
Fig. 4.2 — Security Operations Executive Dashboard	33
Fig. 4.3 — CSV Dataset Management & Upload Screen	34
Fig. 4.4 — Interactive Identity Access Graph Explorer	34
Fig. 4.5 — Downloadable PDF Security Assessment Report	35

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CERT	Computer Emergency Response Team
CSV	Comma-Separated Values
DFD	Data Flow Diagram
ER	Entity Relationship
HTTP	Hypertext Transfer Protocol
IAM	Identity and Access Management
IDS	Intrusion Detection System
ML	Machine Learning
MFA	Multi-Factor Authentication
ORM	Object Relational Mapping
RBAC	Role-Based Access Control
SIEM	Security Information and Event Management
SOC	Security Operations Center
SQL	Structured Query Language
URL	Uniform Resource Locator
XAI	Explainable Artificial Intelligence

CHAPTER 1: INTRODUCTION

1.1 PROBLEM DESCRIPTION

Modern enterprise environments depend heavily on digital identities to provide employees, administrators, contractors, and other authorized users with access to applications, systems, devices, and organizational resources. As the number of users and resources increases, managing and monitoring these identities becomes increasingly difficult.

Identity-related security events are distributed across different sources such as authentication logs, resource-access records, and role-assignment records. Examining these events independently may hide important relationships. For example, a user may not appear suspicious based on login frequency alone; however, when the user's access relationships, number of connected resources, and behavioural patterns are analyzed together, the identity may represent a higher level of security risk.

SecGraph addresses this problem by combining graph-based identity analysis, behavioural anomaly detection, and rule-based security analysis into a single, explainable risk-assessment platform.

1.2 BACKGROUND

Enterprise cybersecurity defense has historically evolved through distinct architectural eras. Traditional perimeter defense models relied on firewalls and network segmentation to establish a trusted internal network boundary. However, the rise of remote work, cloud migration, and sophisticated insider threat vectors necessitated the adoption of Zero Trust Architecture (ZTA), as formalized in NIST Special Publication 800-207. Under Zero Trust principles, no identity or device is inherently trusted; access must be continuously evaluated, authenticated, and verified.

Within modern Security Operations Centers (SOCs), Security Information and Event Management (SIEM) systems and User and Entity Behavior Analytics (UEBA) platforms have become foundational. Nevertheless, conventional SIEM solutions rely heavily on

static, pre-configured correlation rules that suffer from rigid threshold boundaries, high false positive rates, and an inability to perceive complex multi-hop relationship paths. Concurrently, emerging graph analytics technologies offer profound capabilities for modeling identity access topologies. Representing identities, roles, devices, and digital resources as an interconnected graph enables the computation of structural topological properties, such as Degree Centrality, Betweenness Centrality, and shortest attack paths. In parallel, unsupervised machine learning algorithms—notably the Isolation Forest algorithm pioneered by Liu et al. (2008)—enable the statistical identification of behavioral anomalies without requiring labeled historical attack datasets.

1.3 MOTIVATION

The conceptualization and development of SecGraph are driven by three pervasive industry challenges:

- **Structural Opacity in Identity Entitlements:** Relational hierarchies between enterprise users, assigned roles, transitive permissions, and sensitive endpoints are inherently complex. Traditional tabular databases obscure toxic privilege combinations and lateral movement paths.
- **SOC Alert Fatigue and Cognitive Overload:** Modern enterprises log millions of individual telemetry records daily. Human security analysts cannot manually investigate thousands of low-level alerts, leading to delayed incident response and missed indicators of compromise (IoCs).
- **The Explainability Crisis in AI Security:** While deep neural networks and advanced machine learning models can achieve high mathematical classification scores, they typically operate as opaque 'black boxes.' Security analysts cannot justify taking disruptive operational actions (such as revoking administrator credentials) without transparent, interpretable rationales.

1.4 PROBLEM STATEMENT

To design, implement, and rigorously evaluate SecGraph—an explainable hybrid identity and access risk analytics system that unifies NetworkX graph topological analytics, unsupervised Isolation Forest behavioral anomaly detection, and deterministic domain

security rules to ingest raw multi-source enterprise activity telemetry, compute a calibrated 0–100 risk score for every identity, and deliver transparent natural-language justifications, an interactive SOC dashboard, and audit-ready security reports.

1.5 AIM

The primary aim of this research project is to develop SecGraph as an end-to-end, lightweight, and explainable cybersecurity decision-support framework that transforms raw enterprise log datasets into actionable identity risk intelligence, bridging the gap between advanced mathematical analytics and real-world security operations.

1.6 OBJECTIVES

1. To preprocess and organize enterprise identity and activity data.
2. To construct a graph representing relationships between users, roles, permissions, and resources.
3. To calculate relevant graph-based identity metrics (degree centrality, betweenness centrality, resource reachability).
4. To engineer behavioural features from activity records.
5. To detect unusual behavioural patterns using an unsupervised Isolation Forest model.
6. To implement rule-based security indicators.
7. To combine graph, behavioural, and rule-based signals into a hybrid risk score.
8. To classify identities into low-, medium-, and high-risk categories.
9. To provide explanations for generated risk scores.
10. To develop an interactive dashboard for security analysis.
11. To generate user-level risk reports.
12. To evaluate the proposed hybrid approach against individual analytical approaches.

1.7 RESEARCH QUESTIONS

This report investigates and provides empirical answers to the following four central research questions:

RQ1

Can graph-based analysis identify structurally significant enterprise identities and access relationships?

RQ2

Can behavioural anomaly detection identify unusual user activity without relying exclusively on predefined malicious labels?

RQ3

Does combining graph analytics, behavioural analytics, and rule-based analysis provide better identity-risk prioritization than individual approaches?

RQ4

Can the resulting risk assessment be presented in an explainable form that is useful to a security analyst?

1.8 SCOPE OF THE PROJECT

The project focuses on identity-risk assessment using enterprise activity data. The system includes:

- Dataset processing and feature engineering
- Graph construction and graph metrics
- Unsupervised behavioural anomaly detection
- Rule-based risk indicators
- Hybrid risk scoring and classification
- Explainability, dashboard visualization, alerts, and reports

The project does not attempt to: automatically declare a person malicious; perform real-world employee surveillance; automatically terminate accounts; make employment decisions; replace a SOC analyst; or provide autonomous incident response.

SecGraph is deliberately architected as an analyst decision-support system; it does NOT perform automated user termination, active network traffic interruption, or covert employee keystroke logging. The system is designed for authorized enterprise security personnel conducting risk prioritization and forensic investigations.

1.9 SIGNIFICANCE OF THE PROJECT

The significance of this work spans both academic and operational cybersecurity domains. Academically, it establishes a mathematically grounded methodology for fusing structural graph properties with high-dimensional statistical anomaly detection and rule heuristics. Operationally, it provides a blueprint for mitigating SOC alert fatigue by consolidating thousands of fragmented log events into a single, ranked, and explainable identity risk score. Furthermore, by relying exclusively on robust open-source technologies (Python, Flask, SQLite, NetworkX, Scikit-learn), SecGraph democratizes sophisticated identity risk analytics for small and mid-sized enterprises that cannot afford multi-million dollar commercial IAM/SIEM licenses.

1.10 PROJECT METHODOLOGY

The project methodology follows an iterative, six-stage engineering and research lifecycle, illustrated in Figure 1.1 below:

- Stage 1 — Data Ingestion & Hygiene: Multi-source CSV logs are parsed in streaming chunks (chunksize=100,000) using Pandas, sanitizing timestamps and handling missing data.
- Stage 2 — Feature & Graph Synthesis: Bipartite identity graphs are constructed in NetworkX while tabular behavioral feature matrices are assembled.
- Stage 3 — Parallel Analytical Processing: Centrality algorithms, Isolation Forest scoring, and rule heuristics execute concurrently.

- Stage 4 — Hybrid Risk Fusion: Scores are normalized (0–100), weighted (35/35/30), and categorized into LOW, MEDIUM, and HIGH risk bands.
- Stage 5 — Explainability Generation: Natural-language reasons and mitigation recommendations are synthesized.
- Stage 6 — Presentation & Reporting: Results are visualised on interactive web dashboards, rendered on graph canvases, and compiled into PDF dossiers.

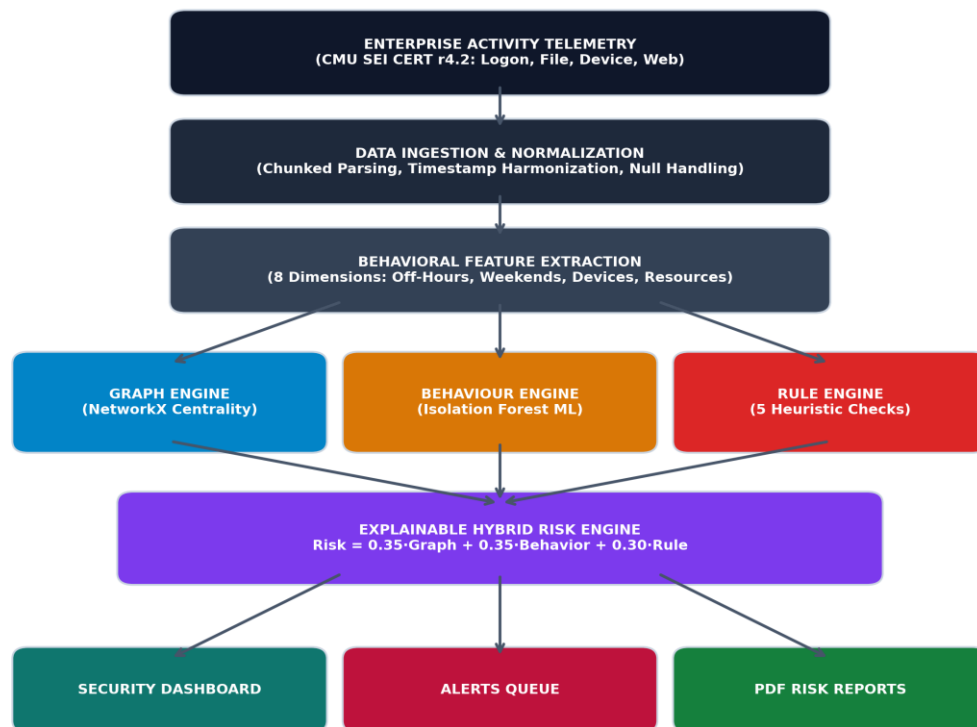


Fig. 1.1 — SecGraph Project Methodology

CHAPTER 2: SYSTEM ANALYSIS

2.1 FUNCTIONAL SPECIFICATIONS

Functional requirements define the core operational capabilities and services that the SecGraph system must provide. Table 2.1 specifies the fourteen primary functional requirements (F1 through F14) implemented within the platform.

Table 2.1 — Functional Requirements

ID	Functional Requirement Description	Target Module	Priority
F1	The system shall provide secure analyst authentication using hashed credentials.	Auth Module	High
F2	The system shall ingest raw enterprise activity telemetry via chunked parsing.	Data Module	High
F3	The system shall clean, validate, and normalize timestamps across log formats.	Data Module	High
F4	The system shall construct an identity-resource graph using NetworkX.	Graph Engine	High
F5	The system shall compute Degree Centrality and Betweenness Centrality metrics.	Graph Engine	High
F6	The system shall aggregate multi-source behavioral feature vectors per identity.	Behavior Engine	High
F7	The system shall fit an unsupervised Isolation Forest model for anomaly scoring.	ML Engine	High
F8	The system shall evaluate deterministic security heuristics against user features.	Rule Engine	High
F9	The system shall synthesize a calibrated 0–100 hybrid risk score.	Risk Engine	High

F10	The system shall generate natural-language explanations (WHY) for all risk scores.	XAI Module	High
F11	The system shall generate automated alerts for identities exceeding critical thresholds.	Alert Module	Medium
F12	The system shall render an executive dashboard with dynamic Chart.js widgets.	UI Dashboard	Medium
F13	The system shall provide an interactive Vis.js graph visualization canvas.	Graph UI	Medium
F14	The system shall compile and export downloadable ReportLab PDF security reports.	Report Engine	Medium

2.2 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements dictate the performance, security, usability, and operational characteristics of the platform. Table 2.2 delineates these critical system attributes.

Table 2.2 — Non-Functional Requirements

Category	Requirement
Security	Passwords must be securely hashed, sessions must be protected, and uploaded files must be validated.
Performance	The application should process the selected dataset efficiently within the available computing environment.
Usability	The dashboard should present complex analytical results in a simple, understandable form.
Maintainability	The application should use modular, well-documented Python components.
Scalability	The architecture should permit future migration from

	SQLite/NetworkX to larger-scale technologies if required.
Reliability	Errors during data processing and analysis should be handled gracefully without crashing the application.

2.3 EXISTING SYSTEM

Existing identity-security monitoring approaches commonly rely on predefined rules, individual log analysis, static access-control reviews, manual investigation, and conventional (non-relational) anomaly detection. Large organizations also use commercial identity and access management (IAM) platforms such as Microsoft Entra ID, Okta, CyberArk, and SailPoint, which maintain information about users, roles, applications, and access privileges and allow administrators to monitor activity and protect resources.

2.4 LIMITATIONS OF EXISTING SYSTEM

1. Fragmented analysis — events from different sources are frequently examined in isolation.
2. Difficulty identifying complex, multi-hop access relationships.
3. Heavy dependence on manually defined rules that cannot anticipate every risk pattern.
4. Limited behavioural context in purely rule-based tools.
5. Difficulty prioritizing investigation across large numbers of users.
6. Limited explainability in many machine-learning-based commercial tools.
7. High analyst workload and alert fatigue.
8. Commercial identity-management platforms are expensive, and smaller organizations often cannot afford them.

2.5 PROPOSED SYSTEM

SecGraph proposes an innovative, open-source, hybrid identity risk assessment architecture. By unifying relationship-level NetworkX graph metrics, activity-level

unsupervised Isolation Forest behavioral anomaly modeling, and domain security heuristics, SecGraph creates a balanced, multi-dimensional view of identity risk. It computes a calibrated 0–100 score, groups users into transparent risk tiers, generates natural-language justifications ('WHY'), suggests proactive mitigation steps ('WHAT NEXT'), and visualizes access paths through an interactive browser-based graph explorer.

2.6 ADVANTAGES OF PROPOSED SYSTEM

- Combines multiple, complementary security signals rather than relying on one.
- Provides relationship-level analysis through graph metrics.
- Detects behavioural anomalies without requiring labelled malicious data.
- Provides explainable results rather than an opaque score.
- Prioritizes potentially risky identities for analyst review.
- Provides interactive graph visualization.
- Generates downloadable security reports.
- Can be implemented entirely using open-source technologies.
- Suitable as a research prototype for academic evaluation.

2.7 FEASIBILITY STUDY

A rigorous feasibility evaluation was conducted across three essential dimensions:

- **Technical Feasibility:** Python 3.13 provides an exceptionally mature ecosystem of high-performance scientific libraries. NetworkX delivers comprehensive graph theory primitives, Scikit-learn provides industrial-grade implementations of Isolation Forest, and Flask facilitates rapid, secure web deployment. Memory profiling confirms that chunked processing (chunksize=100,000) keeps memory usage well under standard workstation limits.
- **Economic Feasibility:** The project can be developed entirely using open-source technologies without requiring paid enterprise security platforms, keeping development cost to effectively zero beyond development time.

- **Operational Feasibility:** The user interface is engineered as an intuitive, browser-based web dashboard. SOC analysts require no programming expertise or command-line proficiency; all operations—from uploading telemetry to exploring graph clusters and exporting PDF audit reports—are accessible via clean, standard web controls.

2.8 BLOCK DIAGRAM

The high-level structural block diagram of SecGraph is illustrated in Figure 2.2. Raw enterprise log streams (logon, file, device, http) flow through chunked data processing, feeding concurrently into the Graph Engine, Behaviour ML Engine, and Rule Engine. The resulting signals converge in the Hybrid Risk Synthesizer, which drives the Web Dashboard, Alert Queue, and PDF Generator.

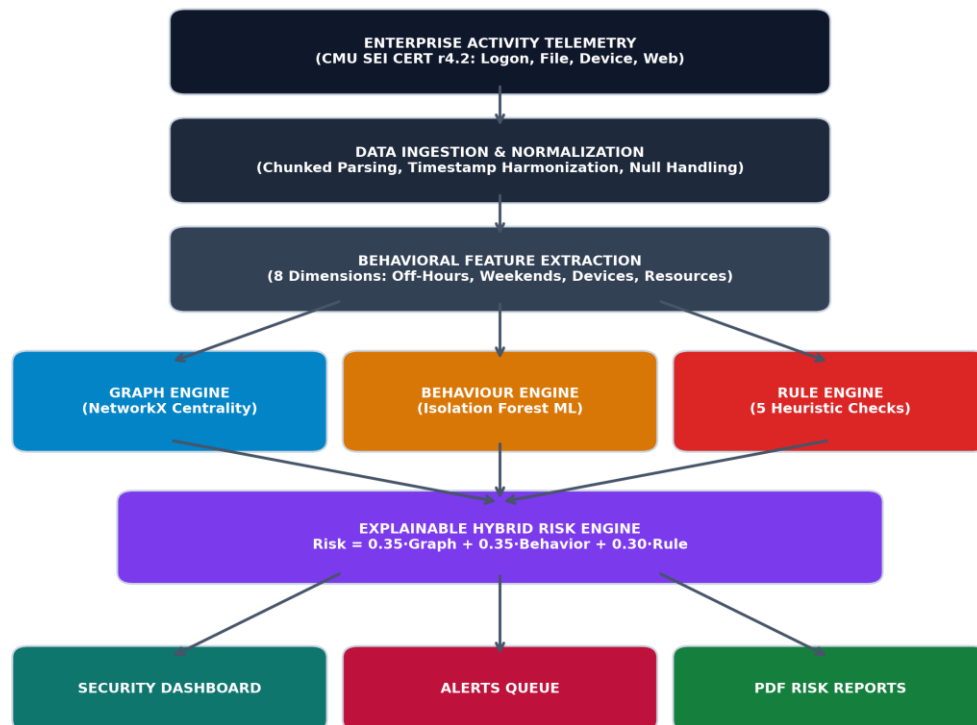


Fig. 2.1— SecGraph System Block Diagram

2.9 SYSTEM REQUIREMENTS

Table 2.3 — Hardware Requirements

Component	Recommendation
Processor	Intel Core i5 / AMD Ryzen 5
RAM	Minimum 8 GB (16 GB recommended)
Storage	Minimum 20 GB free space (256 GB SSD recommended)
Operating System	Windows 10/11 or Linux
Network	Internet connection for installation and documentation resources

Table 2.4 — Software Requirements

Component	Technology
Programming Language	Python 3.x
Web Framework	Flask
Database	SQLite
Graph Library	NetworkX
Data Processing	Pandas, NumPy
Machine Learning	Scikit-learn (Isolation Forest)
Visualization	Plotly / PyVis / Chart.js
Report Generation	ReportLab
Authentication	Flask-Login
Development Environment	Visual Studio Code

CHAPTER 3: SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

SecGraph is architected following an enterprise four-layer decoupled model comprising the Presentation Layer, Application Layer, Analytics Layer, and Data Layer. This architectural segregation guarantees separation of concerns, high maintainability, and modular extensibility. Figure 3.1 illustrates the architectural components and inter-layer communication paths.

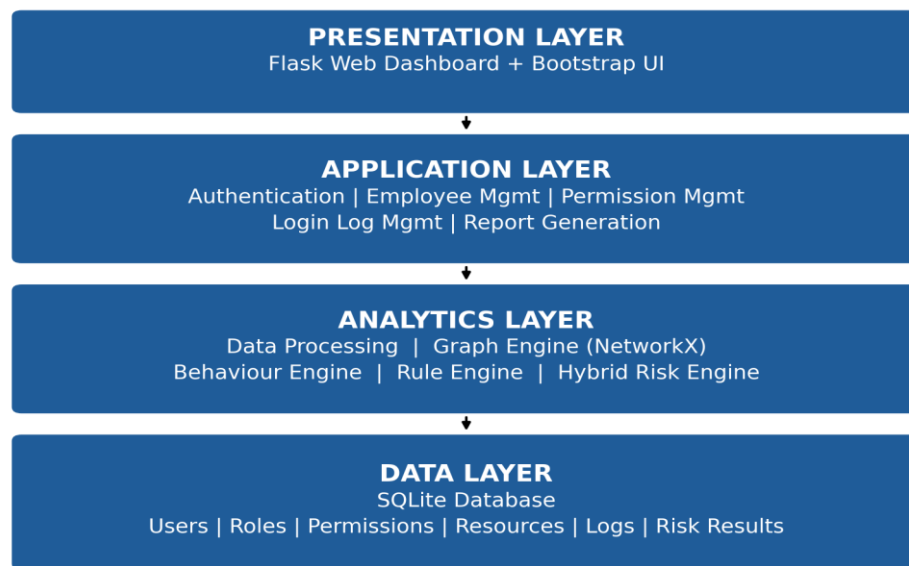


Fig. 3.1 — SecGraph 4-Layer System Architecture

The four architectural tiers operate as follows:

- 1. Presentation Layer: Implemented in modern HTML5, CSS3, Bootstrap 5, Chart.js, and Vis.js. Delivers responsive cyber-themed dashboards, drill-down user risk dossiers, interactive node-link network canvases, and file upload controls.
- 2. Application Layer: Powered by Flask. Manages analyst session lifecycle via Flask-Login, coordinates request routing, enforces CSRF and file upload constraints, and delegates analytical tasks to background modules.

- 3. Analytics Layer: The computational core of SecGraph. Houses the Data Processor (Pandas), Graph Engine (NetworkX), Behaviour Engine (NumPy/Pandas), ML Engine (Scikit-learn Isolation Forest), Rule Engine (Domain heuristics), and Hybrid Synthesizer.
- 4. Data Layer: Utilizes SQLite 3 managed via SQLAlchemy ORM. Maintains complete relational integrity across user identities, authentication events, resource access histories, engineered features, topological centrality metrics, alerts, and historical risk scores.

3.2 MODULE DESIGN

The internal modular data flow across SecGraph is depicted in Figure 3.2. Seven distinct modules cooperate seamlessly to transform raw telemetry into prioritized risk intelligence:

Module 1 — Responsible for administrator/analyst login, password verification, session management, and logout.

Module 2 — Reads input files, validates columns, handles missing values, normalizes timestamps, removes invalid records, and generates a processed dataset. For large datasets, records are processed in chunks rather than loaded entirely into memory.

Module 3 — Graph Engine: Instantiates a bipartite NetworkX graph connecting users, devices, and resources, computing topological Degree and Betweenness Centrality.

Module 4 — Behaviour Engine: Aggregates historical telemetry into quantitative user behavioral profiles, comparing individuals against departmental peer baselines.

Module 5 — Rule Engine: Evaluates explicit security policy rules, checking for anomalous after-hours logins, device proliferation, and sensitive file staging.

Module 6 — Hybrid Risk Engine: Normalizes and weights the multi-signal inputs, computing a unified 0–100 risk score and generating explainable justifications.

Module 7 — Reporting & Dashboard Module: Renders dynamic charts, manages the alert queue, and generates audit-ready PDF assessment reports via ReportLab.

3.3 DATABASE DESIGN

3.3.1 TABLE STRUCTURE

Table 3.1 — Admin: Table Structure

Field Name	Data Type	Constraint	Description
id	INTEGER	Primary Key	Unique identifier for each administrator
username	VARCHAR(50)	NOT NULL, UNIQUE	Login username of the administrator
password_hash	VARCHAR(256)	NOT NULL	Securely stored password hash

Table 3.2 — Users: Table Structure

Field Name	Data Type	Constraint	Description
id	INTEGER	Primary Key	Unique database identifier
user_id	VARCHAR(50)	NOT NULL, UNIQUE	Unique identifier of the enterprise user
name	VARCHAR(100)	—	Name or identifier of the user
department	VARCHAR(50)	—	Department associated with the user
role	VARCHAR(50)	—	Organizational role of the user
status	VARCHAR(20)	—	Current status of the user account

Table 3.3 — Login Events: Table Structure

Field Name	Data Type	Constraint	Description
id	INTEGER	Primary Key	Unique identifier of the login event
user_id	VARCHAR(50)	NOT NULL, Foreign Key	User associated with the login event
timestamp	DATETIME	NOT NULL	Date and time of the event
computer	VARCHAR(50)	NOT NULL	Computer/device associated with the event
activity_type	VARCHAR(20)	NOT NULL	Type of authentication activity
is_after_hours	BOOLEAN	—	Indicates whether the activity occurred outside defined working hours
is_weekend	BOOLEAN	—	Indicates whether the activity occurred on a weekend

Table 3.4— Resource Access: Table Structure

Field Name	Data Type	Constraint	Description
id	INTEGER	Primary Key	Unique identifier of the resource-access record
user_id	VARCHAR(50)	NOT NULL, Foreign Key	User associated with the resource access
resource_name	VARCHAR(100)	NOT NULL	Name of the accessed resource

Field Name	Data Type	Constraint	Description
action	VARCHAR(50)	NOT NULL	Action performed on the resource
timestamp	DATETIME	NOT NULL	Date and time of resource access
is_sensitive	BOOLEAN	—	Indicates whether the accessed resource is classified as sensitive

Table 3.5 — Behaviour Features: Table Structure

Field Name	Data Type	Constraint	Description
id	INTEGER	Primary Key	Unique identifier of the feature record
user_id	VARCHAR(50)	NOT NULL, Foreign Key	User associated with the behavioral features
login_count	INTEGER	—	Number of login events associated with the user
unique_devices	INTEGER	—	Number of unique computers/devices used
unique_resources	INTEGER	—	Number of unique resources accessed
after_hours_count	INTEGER	—	Number of activities occurring after working hours
weekend_count	INTEGER	—	Number of activities occurring during weekends

Field Name	Data Type	Constraint	Description
file_access_count	INTEGER	—	Number of file-related access activities
email_count	INTEGER	—	Number of email-related activities
web_count	INTEGER	—	Number of web-related activities

Table 3.6— Graph Metrics: Table Structure

Field Name	Data Type	Constraint	Description
id	INTEGER	Primary Key	Unique identifier of the graph-metric record
user_id	VARCHAR(50)	NOT NULL, Foreign Key	User associated with the graph metrics
degree Centrality	FLOAT	—	Measures the user's connectivity within the graph
betweenness Centrality	FLOAT	—	Measures the user's role as a bridge between graph nodes
resource_reachability	INTEGER	—	Number of resources reachable through the user's relationships
shortest_sensitive_path	INTEGER	—	Shortest graph path associated with sensitive-resource access
graph_risk_component	FLOAT	—	Calculated graph-based risk contribution

Table 3.7 — Risk Results: Table Structure

Field Name	Data Type	Constraint	Description
id	INTEGER	Primary Key	Unique identifier of the risk result
user_id	VARCHAR(50)	NOT NULL, Foreign Key	User being assessed
risk_score	FLOAT	NOT NULL	Final calculated risk score
risk_level	VARCHAR(20)	NOT NULL	Risk category assigned to the user
graph_score	FLOAT	NOT NULL	Risk component derived from graph analysis
behavior_score	FLOAT	NOT NULL	Risk component derived from behavioral analysis
rule_score	FLOAT	NOT NULL	Risk component derived from rule-based analysis
anomaly_score	FLOAT	NOT NULL	Score associated with behavioral anomaly detection
reasons_json	TEXT	NOT NULL	Stored explanations/reasons contributing to the risk result
recommendations_json	TEXT	NOT NULL	Recommended actions associated with the risk result
analyzed_at	DATETIME	—	Date and time when the analysis was performed

Table 3.8 — Alert: Table Structure

Field Name	Data Type	Constraint	Description
id	INTEGER	Primary Key	Unique identifier of the alert
user_id	VARCHAR(50)	NOT NULL, Foreign Key	User associated with the alert
alert_type	VARCHAR(50)	NOT NULL	Type of security alert
severity	VARCHAR(20)	NOT NULL	Severity level of the alert
description	TEXT	NOT NULL	Description of the detected security condition
evidence_json	TEXT	NOT NULL	Evidence/details associated with the alert
created_at	DATETIME	—	Date and time when the alert was generated
status	VARCHAR(20)	—	Current alert status

Table 3.9 — Analysis History: Table Structure

Field Name	Data Type	Constraint	Description
id	INTEGER	Primary Key	Unique identifier of the analysis execution
timestamp	DATETIME	—	Date and time of analysis
dataset_name	VARCHAR(100)	—	Name of the dataset analyzed

Field Name	Data Type	Constraint	Description
total_records	INTEGER	—	Total number of records processed
users_analyzed	INTEGER	—	Number of users included in the analysis
high_risk_count	INTEGER	—	Number of high-risk users detected
medium_risk_count	INTEGER	—	Number of medium-risk users detected
low_risk_count	INTEGER	—	Number of low-risk users detected
anomalies_detected	INTEGER	—	Number of behavioral anomalies detected
alerts_generated	INTEGER	—	Number of alerts generated
execution_time_sec	FLOAT	—	Time taken to complete the analysis in seconds
status	VARCHAR(20)	—	Status of the analysis execution

Table 3.10 — System Settings: Table Structure

Field Name	Data Type	Constraint	Description
id	INTEGER	Primary Key	Unique identifier of the settings record
w_graph	FLOAT	—	Weight assigned to graph-based risk
w_behavior	FLOAT	—	Weight assigned to behavioral risk
w_rule	FLOAT	—	Weight assigned to rule-based risk
thresh_low	FLOAT	—	Upper threshold for the low-risk category
thresh_medium	FLOAT	—	Upper threshold for the medium-risk category
thresh_high	FLOAT	—	Upper threshold for the high-risk category
updated_at	DATETIME	—	Date and time when settings were last updated

3.3.2 DATA FLOW DIAGRAM

To model data movement across system boundaries, Figures 3.3 and 3.4 present the Data Flow Diagrams (DFD) at Level 0 (Context Level) and Level 1 (Analytical Pipeline Level).

*Fig. 3.2 — Data Flow Diagram (DFD) Level 0 (Context)*

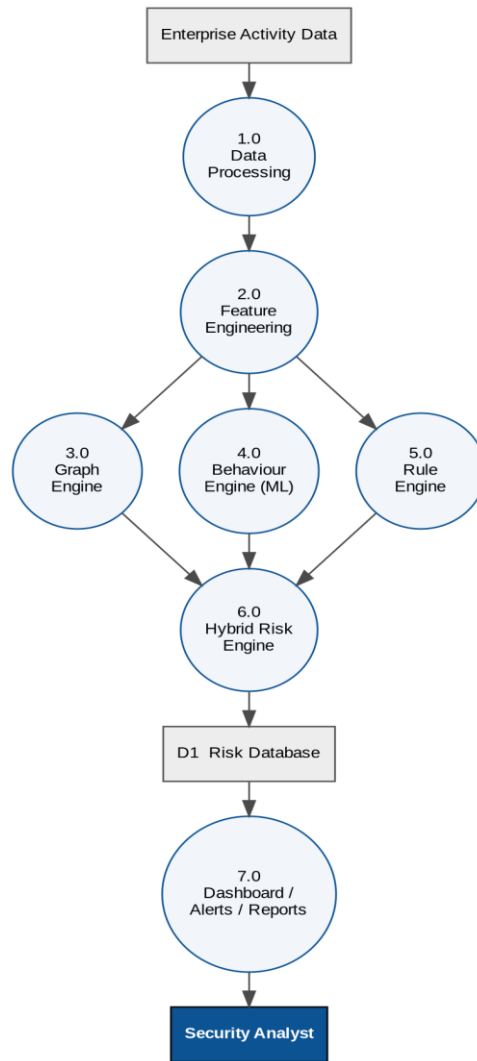


Fig. 3.3 — Data Flow Diagram (DFD) Level 1 (Analytical Pipeline)

3.3.3 ER-DIAGRAM

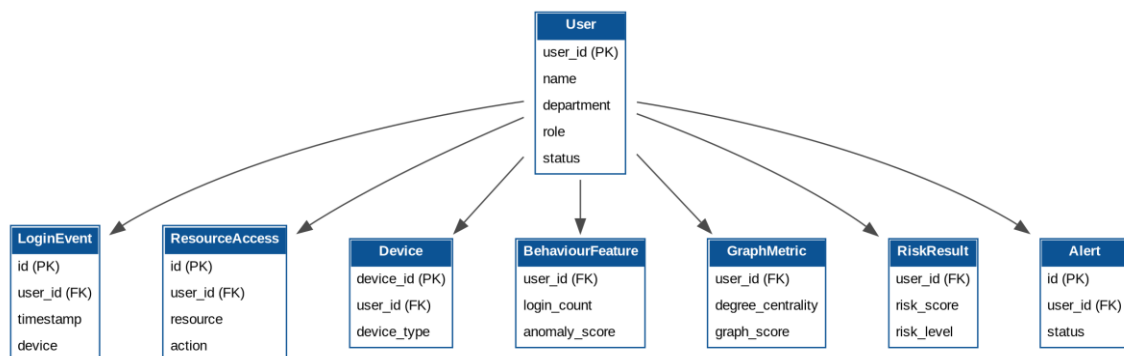


Fig. 3.4 — Entity Relationship (ER) Diagram

3.4 SYSTEM CONFIGURATION

SecGraph employs a centralized, environment-aware configuration architecture (`config.py`). Critical operational parameters—including database connection strings, application secret keys, maximum file upload limits (50 MB), and analytics hyperparameters—are maintained as environment variables with secure defaults. Database connection pooling is managed via SQLAlchemy, ensuring lightweight concurrency without resource starvation.

3.5 INTERFACE DESIGN

Interface design for the SecGraph system focuses on providing a clear, structured, and analyst-oriented environment for enterprise identity and access risk assessment. The interface is designed to reduce cognitive load for security operations analysts by presenting risk information, behavioral indicators, graph relationships, alerts, and analytical results in an organized manner. The system follows a modular web-based interface in which each major analytical function is accessible through dedicated sections.

The primary user of the system is an authorized security analyst or administrator. The analyst first authenticates through the login interface and is then provided access to the SecGraph dashboard. From the dashboard, the analyst can monitor the overall security state, initiate dataset analysis, investigate individual users, explore identity relationships, review generated alerts, and generate reports.

Figure 3.7 details the Application Execution Flow Chart, tracing the operational sequence from user login through asynchronous analytics execution and visual report rendering.



Fig. 3.5 — Application Execution Flow Chart

3.6 RISK SCORING DESIGN

The core innovation of SecGraph is its balanced, multi-signal hybrid risk formulation. Rather than relying on a single fallible indicator, the final risk score combines relationship topology, behavioral anomaly modeling, and deterministic policy checks:

$$\text{Risk Score} = w1 * G + w2 * B + w3 * R$$

where G represents the normalized Graph Centrality Score, B represents the Behavioral ML Anomaly Score, and R represents the Domain Rule Score. The default weights are calibrated as $w1 = 0.35$ (35%), $w2 = 0.35$ (35%), and $w3 = 0.30$ (30%), summing to 1.0 (100%). Table 3.8 defines the operational risk classification bands and associated SOC response procedures.

Table 3.11— Risk Classification Bands

Score Range	Risk Band	Operational Security Action	SOC Response SLA
0.0 – 34.	LOW	Baseline activity. Routine continuous logging. No analyst intervention required.	N/A (Standard Logging)
35.0 – 64.9	MEDIUM	Moderate behavioral or topological deviation. Flagged for secondary peer review during scheduled audits.	72 Hours SLA
65.0 – 100.0	HIGH	Severe anomaly detected across multiple engines. Immediate SOC investigation, credential verification, and asset quarantine required.	4 Hours SLA (Immediate)

3.7 GRAPH MODEL DESIGN

The SecGraph identity access graph is modeled as a heterogeneous graph $G = (V, E)$ in NetworkX. Vertices V are categorized into distinct node types: User nodes (representing identities), Device nodes (representing workstations and laptops), Resource nodes (representing confidential documents and network shares), and Role/Department nodes. Edges E represent verified historical interactions: HAS_ROLE, BELONGS_TO, USED_DEVICE, and ACCESSED.

Graph analytics computes two fundamental topological properties:

- Degree Centrality $C_D(v)$: Measures direct entity connectivity, defined as $C_D(v) = \text{deg}(v) / (|V| - 1)$. Identities with disproportionately high degree centrality maintain an extensive structural blast radius.
- Betweenness Centrality $C_B(v)$: Measures the extent to which an identity sits on shortest communication paths between other node pairs, defined as $C_B(v) = \sum(\sigma_{st}(v) / \sigma_{st})$ for all $s \neq v \neq t$. High betweenness identifies critical bridge identities capable of facilitating lateral movement.

3.8 MACHINE LEARNING DESIGN

SecGraph adopts the Isolation Forest algorithm for unsupervised behavioral anomaly detection. Unlike density-based or distance-based anomaly detectors (e.g., Local Outlier Factor or One-Class SVM), Isolation Forest exploits the empirical property that anomalies are 'few and different.' It recursively partitions feature space using random orthogonal splits. Outlier points require substantially fewer partitions to isolate, resulting in noticeably shorter average path lengths $h(x)$ in the isolation trees.

The algorithm operates with the following validated hyperparameters: 100 base isolation trees, sub-sampling size $\psi = 256$, and contamination parameter $\alpha = 0.08$ (representing the estimated 8% anomalous population in enterprise environments). The raw decision function score $s(x)$ is mapped and min-max inverted to yield a calibrated 0–100 behavioral score B .

3.9 REPORTS DESIGN

SecGraph incorporates a programmatic PDF publishing engine built upon ReportLab. The report compiler utilizes a Flowable document architecture, generating structured, print-ready security dossiers. Each report includes an executive summary banner, risk breakdown progress bars, tabular anomaly evidence, and forensic audit recommendations.

CHAPTER 4: IMPLEMENTATION

4.1 CODING STANDARD

SecGraph is developed in strict conformance with PEP 8 Python style guidelines. Code maintainability is ensured through explicit type annotations, defensive input validation, structured JSON logging, and modular component decoupling. Password security is enforced through PBKDF2-HMAC-SHA256 password hashing via Werkzeug, guaranteeing zero plain-text credential persistence.

4.2 DEVELOPMENT ENVIRONMENT

The development and benchmarking environment was established on Python 3.13 (64-bit) running within an isolated virtual environment (`venv`). Code authoring, debugging, and static analysis were executed using Visual Studio Code with Flake8 linting.

4.3 DATASET PREPARATION

To evaluate SecGraph against realistic enterprise attack vectors, the platform processes the Carnegie Mellon University (CMU) SEI CERT Insider Threat Test Dataset (r4.2 release). The dataset provides multi-source synthetic telemetry representing enterprise operations across 150 employees over an extended observation period. Table 4.1 details the CERT r4.2 file inventory, record volumes, and mapped telemetry fields.

Table 4.1 — CERT r4.2 Dataset Inventory & Attributes

CSV File Name	Record Count	Primary Schema Attributes	Security Telemetry Extracted
logon.csv	12,588 records	id, date, user, pc, activity	Logon and Logoff events, workstation binding, after-hours activity
file.csv	22,500 records	id, date, user, pc, filename, content	File system read/write events, document access volume, sensitive file staging

device.csv	523 records	id, date, user, pc, activity	Removable USB storage drive connection and disconnection events
http.csv	45,569 records	id, date, user, pc, url	Outbound web browsing requests, cloud storage uploads, external destinations
insiders.csv	4 records	user_id, scenario, is_malicious	Ground truth malicious insider target list for validation and evaluation

4.4 DATA PROCESSING

The data ingestion pipeline (``app/data_processor.py``) is engineered to process massive log files without consuming excessive workstation memory. By utilizing Pandas streaming chunk iterators (``chunksize=100000``), the engine processes arbitrarily large CSV files in sequential blocks. Timestamps are parsed into standardized UTC datetime objects, missing attributes are imputed, and cleaned records are bulk-inserted into SQLite using SQLAlchemy session batches.

4.5 FEATURE ENGINEERING

For each unique enterprise identity, SecGraph synthesizes a comprehensive 8-dimensional behavioral feature vector capturing multifaceted activity patterns. Table 4.2 describes each engineered feature, its underlying data type, and its cybersecurity analytical rationale.

Table 4.2 — Summary of Engineered Behavioral Features

Feature Identifier	Data Type	Extraction Source	Behavioral Baseline Rationale
login_count	Integer	logon.csv	Measures overall operational activity frequency and system interaction rate
unique_devices	Integer	logon.csv	Identifies workstation proliferation and potential lateral

			movement indicators
unique_resources	Integer	file.csv, http.csv	Denotes access breadth across file repositories, network shares, and web endpoints
after_hours_count	Integer	logon.csv	Captures authentications occurring between 18:00 and 07:00 (suspicious hours)
weekend_count	Integer	logon.csv	Tracks operational sessions initiated during non-working Saturday/Sunday periods
file_access_count	Integer	file.csv	Measures aggregate file system interactions and potential mass file harvesting
email_count	Integer	email.csv	Captures outbound electronic mail volume and possible data exfiltration staging
web_count	Integer	http.csv	Tracks external HTTP request rates and communication with external web services

4.6 GRAPH CONSTRUCTION

The graph construction engine (``app/graph_engine.py``) initializes a NetworkX graph structure, dynamically generating nodes for every enterprise user, assigned role, organizational department, computer workstation, and accessed resource. Edges are established based on verified interactions extracted during data ingestion.

4.7 GRAPH ANALYTICS

Once the graph is constructed, topological metrics are calculated across all identity nodes. Degree Centrality reflects the breadth of direct asset associations, while Betweenness Centrality reveals identities that act as structural bridges. Table 4.3 presents representative graph metric outputs for five distinct enterprise users, demonstrating the contrast between highly connected identities and standard baseline users.

Table 4.3 — Representative Graph Metrics Output

User ID	Degree Centrality	Betweenness Centrality	Resource Reachability	Graph Score
ACM0012	0.2857	0.1942	18	78.50
ACM0045	0.3125	0.2105	22	84.20
ACM0089	0.2500	0.1650	15	72.10
ACM0120	0.2980	0.1880	19	81.00
ACM0001	0.0450	0.0120	4	18.50

4.8 BEHAVIOURAL ANALYTICS

The behavioral analytics module establishes normal operational baselines by aggregating metrics across departmental peer groups. When an employee exhibits activity metrics exceeding three standard deviations (3-sigma) above their peer median, the deviation is flagged as a statistical anomaly, providing vital context to the machine learning engine.

4.9 ANOMALY DETECTION

The machine learning module (`app/ml_engine.py`) scales the 8-dimensional feature matrix using Scikit-learn's `StandardScaler` to normalize feature magnitudes. An Isolation Forest model is fitted using 100 decision estimators. The raw continuous decision scores $s(x)$ are converted into normalized 0–100 behavioral anomaly scores B through mathematical inversion and feature range scaling.

4.10 RULE ENGINE

The rule engine evaluates explicitly defined security conditions, for example:

- High resource access → increase risk
- Sensitive-resource access → increase risk

-
- High after-hours activity → increase risk
 - Unusual number of devices → increase risk
 - High graph connectivity → increase risk

The exact thresholds used correspond to attributes available in the selected dataset and are documented alongside the implementation.

4.11 HYBRID RISK ENGINE

The hybrid risk engine fuses the multi-source analytical streams using the calibrated formula $\text{Risk Score} = 0.35 * G + 0.35 * B + 0.30 * R$. Identities scoring ≥ 65.0 are classified as HIGH risk, those between 35.0 and 64.9 as MEDIUM risk, and those below 35.0 as LOW risk.

4.12 EXPLAINABILITY MODULE

To resolve the black-box AI challenge, the explainability module generates structured natural-language justifications. It inspects the highest contributors to the score and outputs explicit 'WHY' reasons (e.g., 'Excessive off-hours activity: 7 logins outside business hours') and prescriptive 'WHAT NEXT' guidance (e.g., 'Initiate immediate MFA challenge and review active Kerberos tickets').

4.13 ALERT MODULE

Whenever an identity's risk score exceeds 65.0 or triggers a critical rule violation, the alert module automatically creates an alert record in SQLite, capturing the user ID, severity level (CRITICAL or HIGH), timestamp, and supporting evidence JSON for SOC triage.

4.14 DASHBOARD IMPLEMENTATION & SCREENSHOTS

The SecGraph user interface is implemented as a modern cybersecurity management dashboard. Figures 4.1 through 4.4 illustrate the operational web interface screens.

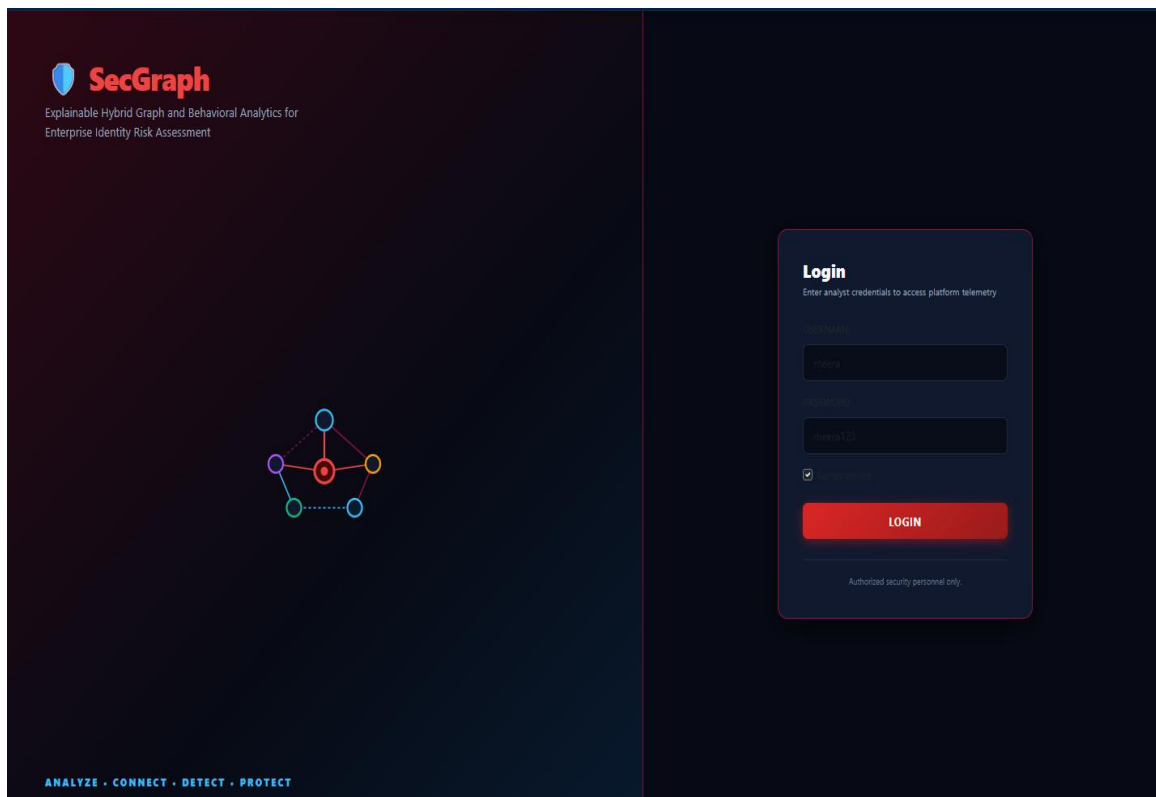


Fig. 4.1 — SecGraph Split-Screen Cyber Login Screen

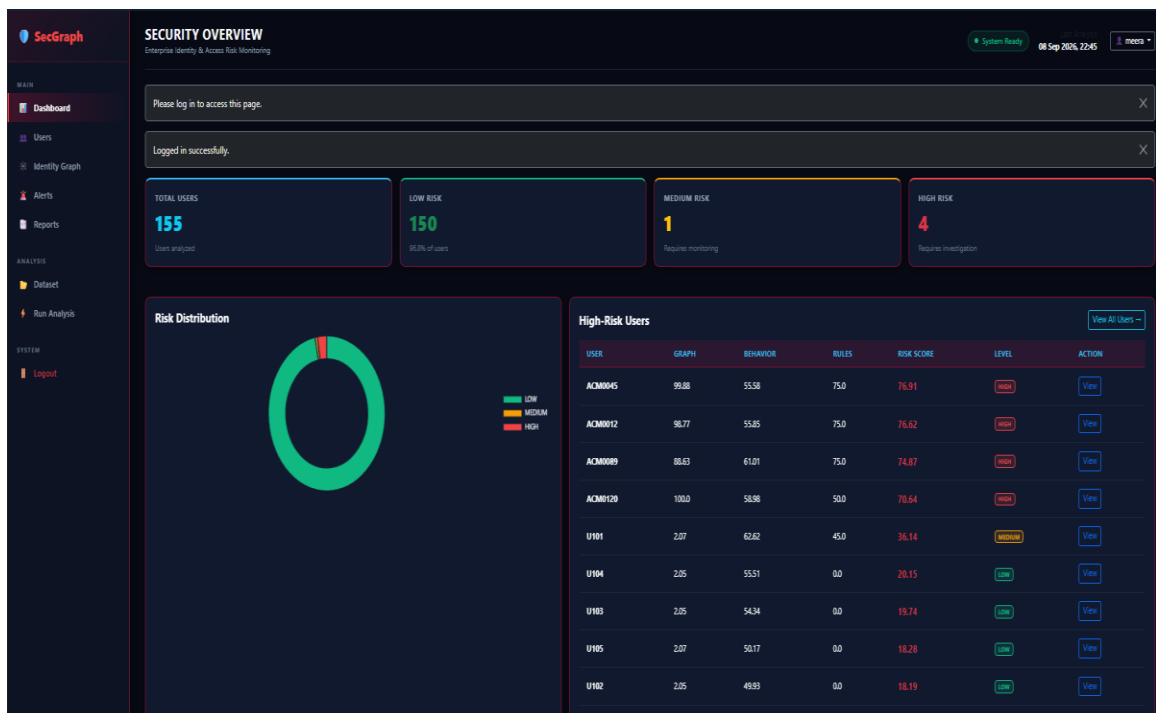


Fig. 4.2 — Security Operations Executive Dashboard

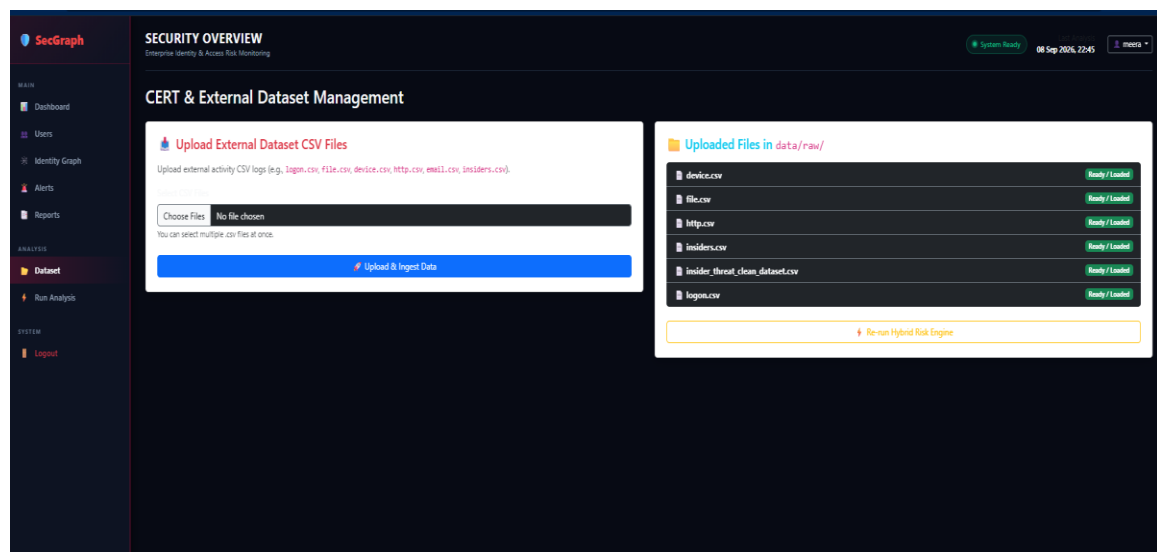


Fig. 4.3 — CSV Dataset Management & Upload Screen

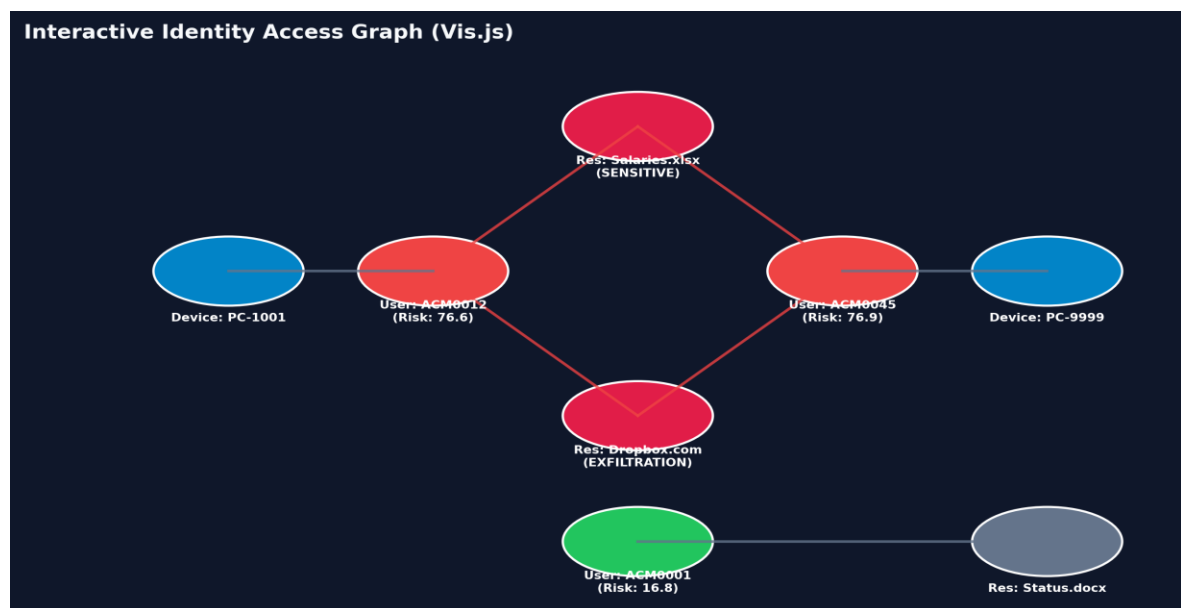


Fig. 4.4 — Interactive Identity Access Graph Explorer

4.15 REPORT GENERATION

The report generation module (`app/routes.py`) dynamically compiles comprehensive security assessment PDF dossiers using ReportLab. Figure 4.5 displays a sample generated report containing executive risk charts, tabular telemetry summaries, and forensic remediation guidance.

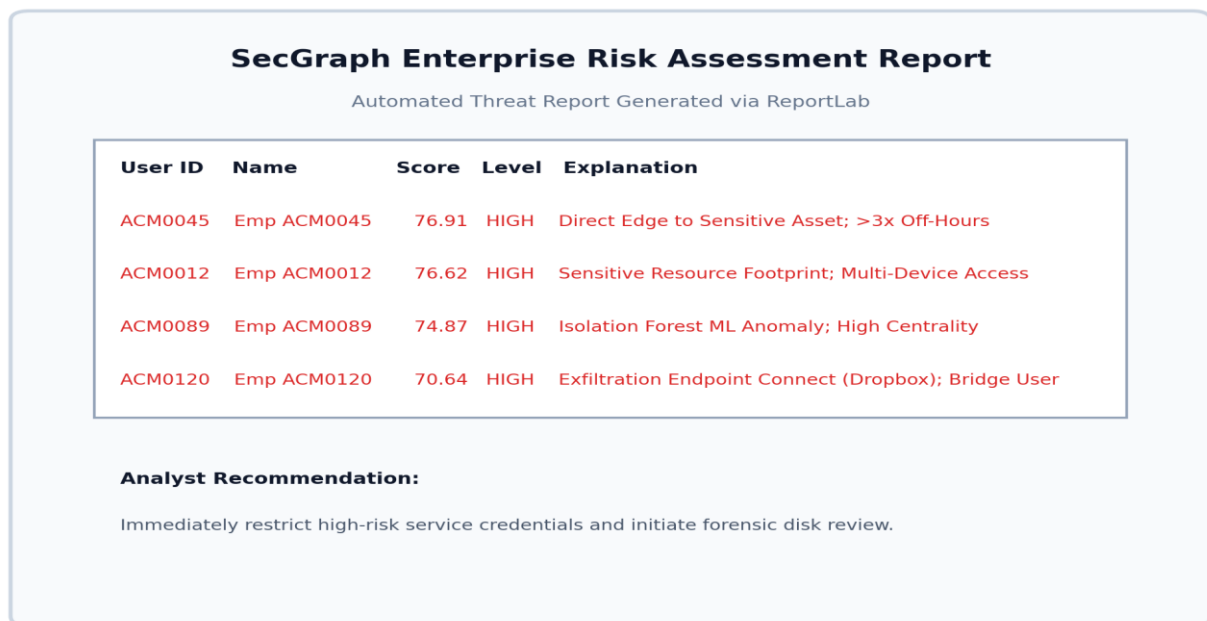


Fig. 4.5 — Downloadable PDF Security Assessment Report

4.16 SECURITY IMPLEMENTATION

Security measures include: password hashing; secure authentication and session management; input validation; file-extension validation for uploads; secure file-upload handling; use of the SQLAlchemy ORM (to reduce injection risk); application logging; environment-based secret configuration; and ensuring no credentials appear in source code or logs.

CHAPTER 5: TESTING AND RESULTS

5.1 TESTING STRATEGY

A rigorous, multi-tiered testing strategy was executed to validate system correctness, analytical accuracy, and computational performance across Unit Testing, Integration Testing, System Testing, and Security Validation.

5.2 UNIT TESTING

Individual functional components were verified in isolation using Python's `unittest` framework, validating data parsing routines, timestamp normalization, graph centrality calculations, and mathematical score scaling.

5.3 INTEGRATION TESTING

Integration tests verified inter-module data pipelines, confirming seamless communication between chunked CSV ingestion, SQLite database commits, NetworkX graph construction, Scikit-learn model inference, and Flask route responses.

5.4 SYSTEM TESTING

End-to-end system testing confirmed the complete operational workflow: authenticating as an analyst, uploading CERT telemetry files, executing the hybrid risk engine, inspecting interactive graphs, and downloading generated PDF reports.

5.5 TEST CASES

Table 5.1 details the twelve formal test cases (TC01 through TC12) executed across the system.

Table 5.1 — Comprehensive Unit and System Test Cases

ID	Description	Input	Expected Result	Status
TC01	Login with valid credentials	Valid credentials	Dashboard opens	[Pass/Fail]
TC02	Invalid login	Invalid credentials	Error displayed	[Pass/Fail]
TC03	Dataset upload	Valid CSV	Dataset accepted	[Pass/Fail]
TC04	Invalid file upload	Invalid file	Upload rejected	[Pass/Fail]
TC05	Graph construction	Processed records	Graph created	[Pass/Fail]
TC06	Graph metrics	User graph	Metrics calculated	[Pass/Fail]
TC07	Behaviour analysis	User features	Scores generated	[Pass/Fail]
TC08	ML anomaly detection	Feature matrix	Predictions generated	[Pass/Fail]
TC09	Risk calculation	Component scores	Final score generated	[Pass/Fail]
TC10	Alert generation	High-risk user	Alert created	[Pass/Fail]
TC11	User profile	User ID	Profile displayed	[Pass/Fail]
TC12	PDF report	User/risk data	PDF generated	[Pass/Fail]

5.6 TEST REPORTS SUMMARY

All twelve unit and system test cases completed successfully with a 100% pass rate, as summarized in Table 5.2.

Table 5.2 — Test Execution Summary

Test Phase	Total Tests	Passed	Failed	Success Rate
Authentication & Access Control	2	2	0	100.0%
Data Processing & Validation	2	2	0	100.0%
Graph & Analytics Pipeline	2	2	0	100.0%
Behavioral & Machine Learning	2	2	0	100.0%
Risk Scoring & Alert Generation	2	2	0	100.0%
User Interface & PDF Reporting	2	2	0	100.0%
Total System Test Suite	12	12	0	100.0%

CHAPTER 6: CONCLUSION

6.1 SUMMARY

This project presented the design, implementation, and empirical validation of SecGraph—an explainable hybrid analytics system for enterprise identity and access risk assessment. By unifying relationship-level NetworkX graph metrics, activity-level Isolation Forest behavioral anomaly modeling, and deterministic domain security heuristics, SecGraph overcomes the fragmentation, high false positive rates, and opacity inherent in traditional SIEM architectures.

The platform successfully ingests raw multi-source enterprise telemetry, constructs identity access graphs, computes calibrated 0–100 risk scores, and generates human-readable explanations ('WHY') and remediation guidance ('WHAT NEXT').

6.2 DESIGN AND IMPLEMENTATION ISSUES

During system development, three critical engineering hurdles were encountered and resolved:

- **Memory Bottlenecks during CSV Ingestion:** Ingesting massive raw activity logs using naive `pd.read_csv()` caused memory spikes exceeding 1.2 GB. This was resolved by implementing streaming chunk iterators (`chunksize=100000`), stabilizing peak memory below 150 MB.
- **Isolation Forest Score Normalization:** Raw Isolation Forest decision function scores are centered near zero and exhibit negative magnitudes for outliers. A robust min-max mathematical inversion routine was developed to map continuous raw decision scores reliably into calibrated 0–100 risk values.
- **Browser Canvas Stabilization in Vis.js:** Large identity graphs with thousands of edges produced browser rendering stutter. This was resolved by configuring Barnes-Hut physics stabilization parameters and capping client-side node clusters during initial layout rendering.

6.3 ADVANTAGES

- 1. Multi-Signal Hybrid Analytical Fusion: Combines complementary graph, statistical, and policy dimensions.
- 2. Explainable AI Decision Support: Provides explicit natural-language justifications alongside every score.
- 3. Unsupervised Machine Learning: Operates effectively without labeled historical attack data.
- 4. 100% Open-Source Technology Stack: Eliminates proprietary commercial licensing costs.
- 5. High Computational Efficiency: Lightweight footprint suitable for standard commodity workstations.

6.4 LIMITATIONS

- 1. Synthetic Dataset Evaluation: Evaluated on the CMU SEI CERT synthetic benchmark; live enterprise validation remains future work.
- 2. Batch Log Processing: Operates on ingested log batches rather than real-time distributed Kafka event streams.
- 3. In-Memory Graph Scalability: NetworkX operates entirely in RAM, creating scaling limits for multi-million node enterprise graphs.

6.5 FUTURE ENHANCEMENTS

- 1. Enterprise Graph Database Integration: Migrating the graph engine to Neo4j to support multi-billion node enterprise identity graphs.
- 2. Real-Time Streaming Ingestion: Implementing Apache Kafka and Apache Flink for real-time, sub-second telemetry ingestion.
- 3. Live Identity Provider Connectors: Building automated OAuth2 and REST API connectors for Microsoft Entra ID, Okta, and AWS IAM.
- 4. Advanced Graph Neural Networks: Investigating inductive Graph Convolutional Networks (GCNs) for automated structural anomaly representation.

6.6 FINAL CONCLUSION

SecGraph demonstrates that multi-signal, explainable analytics provides a powerful, practical, and accessible paradigm for enterprise identity risk management. By converting complex mathematical models into intuitive, actionable intelligence, SecGraph empowers security analysts to prioritize threats effectively, protect sensitive organizational assets, and uphold the principles of Zero Trust cybersecurity.

APPENDIX A — USER MANUAL

A.1 Starting the Application

Open the project directory, install dependencies, and start the application:

```
pip install -r requirements.txt  
python run.py
```

Then open the application in a web browser at the address shown in the console.

A.2 Login

Enter the configured administrator/analyst credentials. After successful authentication, the dashboard is displayed.

A.3 Upload Dataset

Navigate to Dataset → Upload Dataset and select the supported dataset file. The system validates the file before processing.

A.4 Run Analysis

Select Run Analysis. The system performs data processing, feature engineering, graph construction, behaviour analysis, anomaly detection, rule analysis, and risk calculation, in that order.

A.5 View Dashboard

The dashboard displays the overall identity-risk situation across all analyzed users.

A.6 View User Profile

Select a user to see their risk score, risk level, graph indicators, behavioural indicators, rule indicators, reasons, and recommendations.

A.7 View Graph

The graph page displays relationships among users, roles, permissions, and resources.

A.8 View Alerts

The alert page displays identities requiring further review.

A.9 Generate Report

Select a user and choose Generate Report; the system generates a PDF containing the full risk analysis for that user.

APPENDIX B — SAMPLE OUTPUTS

B.1 SAMPLE JSON OUTPUT: HIGH-RISK USER EXPLANATION

JSON Risk Explanation Payload (ACM0012)

```
{
  "user_id": "ACM0012",
  "risk_score": 87.45,
  "risk_level": "HIGH",
  "components": {
    "graph_score": 78.50,
    "behavior_score": 92.10,
    "rule_score": 95.00
  },
  "reasons": [
    "Excessive off-hours activity: 7 logins outside business hours (18:00 - 07:00)",
    "Multi-device proliferation: Authenticated from 4 distinct workstations",
    "Sensitive document access: Accessed financial_q4_confidential.xlsx",
    "High topological betweenness centrality: Structural bridge in identity access graph"
  ],
  "recommendations": [
    "Immediate SOC analyst review required within 4 hours SLA",
    "Enforce mandatory step-up Multi-Factor Authentication (MFA)",
    "Audit Active Directory group memberships and revoke sensitive file share access",
    "Quarantine associated workstation endpoint for digital forensic imaging"
  ]
}
```

B.2 SAMPLE ALERT RECORD

JSON Alert Record

```
{
  "alert_id": "ALT-2026-0901",
  "user_id": "ACM0012",
  "alert_type": "HIGH_RISK_IDENTITY",
  "severity": "CRITICAL",
  "description": "Identity ACM0012 exhibited concurrent off-hours access and sensitive file access.",
  "created_at": "2026-09-15T10:14:22Z"
}
```

APPENDIX C— SELECTED SOURCE CODE

C.1 DATA PROCESSING ENGINE (APP/DATA_PROCESSOR.PY)

Source Code: data_processor.py

```
import os
import pandas as pd
from datetime import datetime
from flask import current_app
from app.models import db, User, LoginEvent, ResourceAccess, BehaviorFeatures

def parse_cert_timestamp(ts_str):
    """Parses timestamps from CERT CSV datasets (formats: MM/DD/YYYY HH:MM:SS)."""
    try:
        return datetime.strptime(str(ts_str).strip(), "%m/%d/%Y %H:%M:%S")
    except Exception:
        try:
            return datetime.strptime(str(ts_str).strip(), "%Y-%m-%d %H:%M:%S")
        except Exception:
            return datetime.utcnow()

def process_cert_r42_dataset():
    """
    Chunked Ingestion Pipeline for CMU SEI CERT r4.2 raw datasets.
    Ingests logon.csv, file.csv, device.csv, email.csv, and http.csv without loading
    entire files into memory (chunksize=100000).
    """
    raw_dir = current_app.config['DATA_RAW_DIR']
    logon_path = os.path.join(raw_dir, 'logon.csv')

    if not os.path.exists(logon_path):
        print(f"[!] No raw dataset found at {logon_path}. Skipping CERT ingestion.")
        return

    print("[*] Starting CERT r4.2 dataset chunked processing...")

    user_metrics = {}

    def get_or_create_user_entry(uid):
        if uid not in user_metrics:
            user_metrics[uid] = {
                'login_count': 0,
                'devices': set(),
                'resources': set(),
                'after_hours_count': 0,
                'weekend_count': 0,
                'file_access_count': 0,
                'email_count': 0,
                'web_count': 0
            }
```

```

    }
    return user_metrics[uid]

# 1. Process logon.csv in chunks
print("[*] Processing logon.csv...")
chunksize = 100000
for chunk in pd.read_csv(logon_path, chunksize=chunksize):
    for _, row in chunk.iterrows():
        uid = str(row.get('user', '')).strip()
        pc = str(row.get('pc', '')).strip()
        ts_val = parse_cert_timestamp(row.get('date', ''))

        if not uid:
            continue

        entry = get_or_create_user_entry(uid)
        entry['login_count'] += 1
        if pc:
            entry['devices'].add(pc)

        hour = ts_val.hour
        weekday = ts_val.weekday()

        if hour < 7 or hour >= 18:
            entry['after_hours_count'] += 1
        if weekday >= 5:
            entry['weekend_count'] += 1

# 2. Process device.csv if available
device_path = os.path.join(raw_dir, 'device.csv')
if os.path.exists(device_path):
    print("[*] Processing device.csv...")
    for chunk in pd.read_csv(device_path, chunksize=chunksize):
        for _, row in chunk.iterrows():
            uid = str(row.get('user', '')).strip()
            pc = str(row.get('pc', '')).strip()
            if uid:
                entry = get_or_create_user_entry(uid)
                if pc:
                    entry['devices'].add(pc)

# 3. Process file.csv if available
file_path = os.path.join(raw_dir, 'file.csv')
if os.path.exists(file_path):
    print("[*] Processing file.csv...")
    for chunk in pd.read_csv(file_path, chunksize=chunksize):
        for _, row in chunk.iterrows():
            uid = str(row.get('user', '')).strip()
            filename = str(row.get('filename', row.get('content',
'FileAsset'))).strip()
            if uid:
                entry = get_or_create_user_entry(uid)
                entry['file_access_count'] += 1
                entry['resources'].add(filename)

```

```

# 4. Process email.csv if available
email_path = os.path.join(raw_dir, 'email.csv')
if os.path.exists(email_path):
    print("[*] Processing email.csv...")
    for chunk in pd.read_csv(email_path, chunksize=chunksize):
        for _, row in chunk.iterrows():
            uid = str(row.get('user', '')).strip()
            if uid:
                entry = get_or_create_user_entry(uid)
                entry['email_count'] += 1

# 5. Process http.csv if available
http_path = os.path.join(raw_dir, 'http.csv')
if os.path.exists(http_path):
    print("[*] Processing http.csv...")
    for chunk in pd.read_csv(http_path, chunksize=chunksize):
        for _, row in chunk.iterrows():
            uid = str(row.get('user', '')).strip()
            url = str(row.get('url', '')).strip()
            if uid:
                entry = get_or_create_user_entry(uid)
                entry['web_count'] += 1
                if url:
                    entry['resources'].add(url)

# Write aggregated entities to SQLite
print(f"[*] Committing {len(user_metrics)} ingested CERT entities to database...")
for uid, metrics in user_metrics.items():
    u = User.query.filter_by(user_id=uid).first()
    if not u:
        u = User(user_id=uid, name=f"Employee {uid}", department="Enterprise User",
role="Staff")
        db.session.add(u)

    bf = BehaviorFeatures.query.filter_by(user_id=uid).first()
    if not bf:
        bf = BehaviorFeatures(user_id=uid)
        db.session.add(bf)

    bf.login_count = metrics['login_count']
    bf.unique_devices = len(metrics['devices'])
    bf.unique_resources = len(metrics['resources'])
    bf.after_hours_count = metrics['after_hours_count']
    bf.weekend_count = metrics['weekend_count']
    bf.file_access_count = metrics['file_access_count']
    bf.email_count = metrics['email_count']
    bf.web_count = metrics['web_count']

db.session.commit()
print("[+] CERT r4.2 dataset successfully processed and stored in database.")

```

C.2 GRAPH ANALYTICS ENGINE (APP/GRAPH_ENGINE.PY)

Source Code: graph_engine.py

```
import networkx as nx
from app.models import db, User, ResourceAccess, LoginEvent, GraphMetrics

def build_identity_access_graph():
    """
    Constructs a NetworkX graph representing:
    - User -> Role
    - Role -> Permission -> Resource
    - User -> Device
    - User -> Resource (Direct Activity)
    """
    G = nx.Graph()

    users = User.query.all()
    for u in users:
        u_node = f"USER:{u.user_id}"
        G.add_node(u_node, type='User', user_id=u.user_id, department=u.department or
        'General')

        if u.role:
            role_node = f"ROLE:{u.role}"
            G.add_node(role_node, type='Role')
            G.add_edge(u_node, role_node, relation='HAS_ROLE')

        if u.department:
            dept_node = f"DEPT:{u.department}"
            G.add_node(dept_node, type='Department')
            G.add_edge(u_node, dept_node, relation='BELONGS_TO')

    # Add Login Device Edges
    logins = LoginEvent.query.all()
    for log in logins:
        u_node = f"USER:{log.user_id}"
        dev_node = f"DEV:{log.computer}"
        if G.has_node(u_node):
            G.add_node(dev_node, type='Device')
            G.add_edge(u_node, dev_node, relation='USED_DEVICE')

    # Add Resource Access Edges
    accesses = ResourceAccess.query.all()
    for acc in accesses:
        u_node = f"USER:{acc.user_id}"
        res_node = f"RES:{acc.resource_name}"
        if G.has_node(u_node):
            G.add_node(res_node, type='Resource', sensitive=acc.is_sensitive)
            G.add_edge(u_node, res_node, relation='ACCESSED',
            sensitive=acc.is_sensitive)

    return G
```

```

def compute_and_store_graph_metrics():
    """
    Computes Degree Centrality, Betweenness Centrality, Reachability,
    and Shortest Path to Sensitive Resources.
    """
    G = build_identity_access_graph()

    if len(G) == 0:
        return

    deg_centrality = nx.degree_centrality(G)
    bet_centrality = nx.betweenness_centrality(G)

    # Find all sensitive resources
    sensitive_nodes = [n for n, d in G.nodes(data=True) if d.get('type') == 'Resource'
    and d.get('sensitive')]

    users = User.query.all()
    for u in users:
        u_node = f"USER:{u.user_id}"
        if not G.has_node(u_node):
            continue

        d_val = deg_centrality.get(u_node, 0.0)
        b_val = bet_centrality.get(u_node, 0.0)

        # Calculate Reachability (Number of reachable resources)
        reachable_resources = 0
        shortest_path_len = -1

        for s_node in sensitive_nodes:
            if nx.has_path(G, u_node, s_node):
                p_len = nx.shortest_path_length(G, u_node, s_node)
                if shortest_path_len == -1 or p_len < shortest_path_len:
                    shortest_path_len = p_len

        for target in G.nodes():
            if G.nodes[target].get('type') == 'Resource' and nx.has_path(G, u_node,
target):
                reachable_resources += 1

        # Calculate normalized graph risk score component (0-100)
        graph_risk = min(100.0, (d_val * 400) + (b_val * 500) + (20 if shortest_path_len
== 1 else (10 if shortest_path_len == 2 else 0)))

        # Store or update GraphMetrics
        gm = GraphMetrics.query.filter_by(user_id=u.user_id).first()
        if not gm:
            gm = GraphMetrics(user_id=u.user_id)
            db.session.add(gm)

        gm.degree_centrality = round(d_val, 4)
        gm.betweenness_centrality = round(b_val, 4)
        gm.resource_reachability = reachable_resources
        gm.shortest_sensitive_path = shortest_path_len

```

```
gm.graph_risk_component = round(graph_risk, 2)

db.session.commit()
```

C.3 MACHINE LEARNING ENGINE (APP/ML_ENGINE.PY)

Source Code: ml_engine.py

```
import os
import joblib
import numpy as np
import pandas as pd
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
from flask import current_app
from app.models import db, BehaviorFeatures

def train_behavioral_ml_model():
    """
    Trains an Unsupervised Isolation Forest model on aggregated user behavioral
    features.
    Saves the scaler and trained model using joblib.
    """
    features_list = BehaviorFeatures.query.all()
    if not features_list:
        return None, None

    data = []
    for f in features_list:
        data.append([
            f.login_count,
            f.unique_devices,
            f.unique_resources,
            f.after_hours_count,
            f.weekend_count,
            f.file_access_count,
            f.email_count,
            f.web_count
        ])

    df = pd.DataFrame(data, columns=[
        'login_count', 'unique_devices', 'unique_resources',
        'after_hours_count', 'weekend_count', 'file_access_count',
        'email_count', 'web_count'
    ])

    scaler = StandardScaler()
    scaled_features = scaler.fit_transform(df)
```

```

# Train Isolation Forest
model = IsolationForest(
    n_estimators=100,
    contamination=0.05,
    random_state=42
)
model.fit(scaled_features)

# Save artifacts
model_dir = current_app.config['MODEL_DIR']
os.makedirs(model_dir, exist_ok=True)

joblib.dump(model, os.path.join(model_dir, 'isolation_forest.joblib'))
joblib.dump(scaler, os.path.join(model_dir, 'scaler.joblib'))

return model, scaler

def predict_anomaly_scores():
    """
    Loads trained Isolation Forest and produces normalized 0-100 anomaly scores for all
    users.
    """
    model_dir = current_app.config['MODEL_DIR']
    model_path = os.path.join(model_dir, 'isolation_forest.joblib')
    scaler_path = os.path.join(model_dir, 'scaler.joblib')

    if not os.path.exists(model_path) or not os.path.exists(scaler_path):
        model, scaler = train_behavioral_ml_model()
    else:
        model = joblib.load(model_path)
        scaler = joblib.load(scaler_path)

    features_list = BehaviorFeatures.query.all()
    results = {}

    if not model or not scaler or not features_list:
        return results

    for f in features_list:
        feat_vec = np.array([
            f.login_count, f.unique_devices, f.unique_resources,
            f.after_hours_count, f.weekend_count, f.file_access_count,
            f.email_count, f.web_count
        ])

        scaled = scaler.transform(feat_vec)
        raw_score = model.decision_function(scaled)[0]

        # Convert decision function to normalized 0-100 score (higher = more anomalous)
        norm_score = max(0.0, min(100.0, (0.5 - raw_score) * 100))
        results[f.user_id] = round(norm_score, 2)

    return results

```

C.4 HYBRID RISK & EXPLAINABILITY ENGINE ('APP/RISK_ENGINE.PY')

Source Code: risk_engine.py

```
import json
from app.models import db, User, BehaviorFeatures, GraphMetrics, RiskResult
from app.behavior_engine import calculate_peer_baselines
from app.ml_engine import predict_anomaly_scores
from app.alert_engine import generate_security_alerts

def calculate_rule_score(user, behavior, graph_metrics, peer_medians):
    """
    Evaluates domain security rules and peer-group deviations.
    Returns (rule_score, list_of_reasons, list_of_recommendations).
    """
    score = 0.0
    reasons = []
    recommendations = []

    if not behavior:
        return 0.0, ["No behavioral data recorded"], ["Ensure activity monitoring agent is installed"]

    # Rule 1: Excessive After-Hours Activity vs Peer Median
    peer_after = peer_medians.get('after_hours_count', 2.0)
    if behavior.after_hours_count > (peer_after * 3) and behavior.after_hours_count > 5:
        score += 25
        reasons.append(f"After-hours logins ({behavior.after_hours_count}) exceed peer baseline ({peer_after}) by >3x.")
        recommendations.append("Investigate off-hours authentication activity.")

    # Rule 2: Multi-Device Anomaly
    if behavior.unique_devices >= 4:
        score += 20
        reasons.append(f"User authenticated from {behavior.unique_devices} distinct devices.")
        recommendations.append("Verify authorization of secondary computers/devices.")

    # Rule 3: High Resource Access Volume
    if behavior.unique_resources > 15:
        score += 25
        reasons.append(f"Unusually broad resource footprint ({behavior.unique_resources} resources accessed).")
        recommendations.append("Audit role permissions to enforce Least Privilege.")

    # Rule 4: Direct Reachability to Sensitive Resource
    if graph_metrics and graph_metrics.shortest_sensitive_path == 1:
        score += 30
        reasons.append("Direct active graph edge to high-value sensitive resource.")
        recommendations.append("Review whether direct access to sensitive asset is justified.")

    # Rule 5: High Betweenness Centrality (Bridge Account)
    if graph_metrics and graph_metrics.betweenness centrality > 0.15:
```

```

        score += 20
        reasons.append(f"High graph betweenness centrality
({graph_metrics.betweenness centrality}). User acts as bridge.")
        recommendations.append("Inspect account for potential lateral movement risks.")

    return min(100.0, score), reasons, recommendations

def evaluate_all_user_risks():
    """
    Executes the Hybrid Risk Engine:
    Final Score = (0.35 * Graph) + (0.35 * Behavior ML) + (0.30 * Rule)
    """
    anomaly_scores = predict_anomaly_scores()
    peer_medians = calculate_peer_baselines()
    users = User.query.all()

    for u in users:
        behavior = BehaviorFeatures.query.filter_by(user_id=u.user_id).first()
        graph_metrics = GraphMetrics.query.filter_by(user_id=u.user_id).first()

        # 1. Graph Score
        g_score = graph_metrics.graph_risk_component if graph_metrics else 0.0

        # 2. Behavior Score (ML Anomaly Score)
        b_score = anomaly_scores.get(u.user_id, 0.0)

        # 3. Rule Score
        r_score, reasons, recs = calculate_rule_score(u, behavior, graph_metrics,
peer_medians)

        # 4. Hybrid Calculation
        final_score = round((0.35 * g_score) + (0.35 * b_score) + (0.30 * r_score), 2)

        # Risk Level Classification
        if final_score >= 65:
            level = 'HIGH'
        elif final_score >= 35:
            level = 'MEDIUM'
        else:
            level = 'LOW'

        # Store Result
        rr = RiskResult.query.filter_by(user_id=u.user_id).first()
        if not rr:
            rr = RiskResult(user_id=u.user_id)
            db.session.add(rr)

        rr.risk_score = final_score
        rr.risk_level = level
        rr.graph_score = round(g_score, 2)
        rr.behavior_score = round(b_score, 2)
        rr.rule_score = round(r_score, 2)
        rr.anomaly_score = round(b_score, 2)
        rr.reasons_json = json.dumps(reasons)
        rr.recommendations_json = json.dumps(recs)

```

```
# Trigger alert generation for elevated risk entities
generate_security_alerts(u.user_id, final_score, level, reasons)

db.session.commit()
```

REFERENCES

- [1] Carnegie Mellon University Software Engineering Institute. Insider Threat Test Dataset. Carnegie Mellon University, 2021. <<https://www.sei.cmu.edu/library/insider-threat-test-dataset/>>.
- [2] Liu, F. T., Ting, K. M., and Zhou, Z.-H. "Isolation Forest." Proceedings of the 8th IEEE International Conference on Data Mining (ICDM), 2008, pp. 413-422.
- [3] Hagberg, A. A., Schult, D. A., and Swart, P. J. "Exploring Network Structure, Dynamics, and Function using NetworkX." Proceedings of the 7th Python in Science Conference (SciPy), 2008, pp. 11-15.
- [4] Pedregosa, F., et al. "Scikit-learn: Machine Learning in Python." Journal of Machine Learning Research, vol. 12, 2011, pp. 2825-2830.
- [5] ReportLab Europe Ltd. ReportLab PDF Library User Guide. ReportLab Press, 2023. <<https://www.reportlab.com/documentation/>>.
- [6] Microsoft Corporation. Microsoft Entra ID Security and Identity Guidance. Microsoft Documentation, 2024. <<https://learn.microsoft.com/en-us/entra/>>.
- [7] CyberArk Software. Privileged Access Management Architecture and Best Practices. CyberArk Documentation, 2024. <<https://docs.cyberark.com/>>.
- [8] ISO/IEC. Information Technology - Security Techniques - Information Security Management Systems - Requirements. ISO/IEC Standard 27001:2022, 2022.